Objektorientierte Programmiertechniken Prof. Dr. Ursula Oesing	Hochschule Bochum TECHNIK WIRTSCHAFT GESUNDHEIT
Übung Nr. 1: Versionierung / Versionsverwaltung (mit Git)	

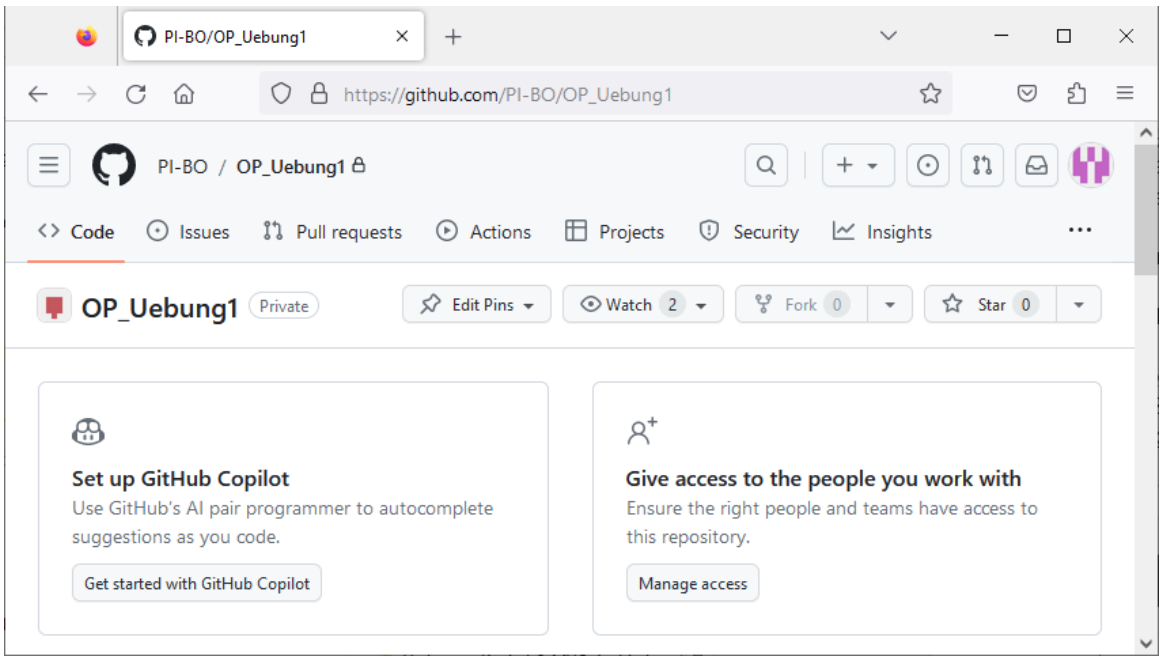
1. Aufgabe zu Git

Üblicherweise arbeiten mehrere Entwickler und Entwicklerinnen an demselben Softwareprodukt. Weiterhin gibt es mehrere Versionen, die parallel gewartet werden müssen, beispielsweise eine Entwicklungsversion, eine Release-Version und eine produktive Version. Man benötigt ein Tool zur Versionierung / Versionsverwaltung, beispielsweise Git. Das probieren Sie jetzt aus.

1.1 Vorbereitungen

Einrichten von Repositories auf dem Server

Benötigt wird ein Git-Repository, hier unter https://github.com, an welchem jeweils zwei Studierende, Entwickler A und Entwickler B, angemeldet sein müssen. (Mit Entwickler A und B sind Entwicklerinnen und Entwickler gemeint!) Sie benötigen im Git-Repository einen Access Token, siehe *KurzanleitungZuGit*



Weiterhin muss EGit in Eclipse der Clients installiert sein, siehe *KurzanleitungZuGit*. Entgegen der Kurzanleitung hat das Repository hier den Namen *OP_Uebung1*. **Sie halten sich im Praktikum an die Angaben der Kurzanleitung.**

Verbinden der Clients mit dem Repository des Servers

Die Studierenden arbeiten in Gruppen von 2 Personen an 2 Clients. Sie öffnen Eclipse. Falls man nicht in der Java-Sicht ist, soll in *Eclipse* die Sicht *Java* ausgewählt werden (Icon *Open Perspective* rechts oben).

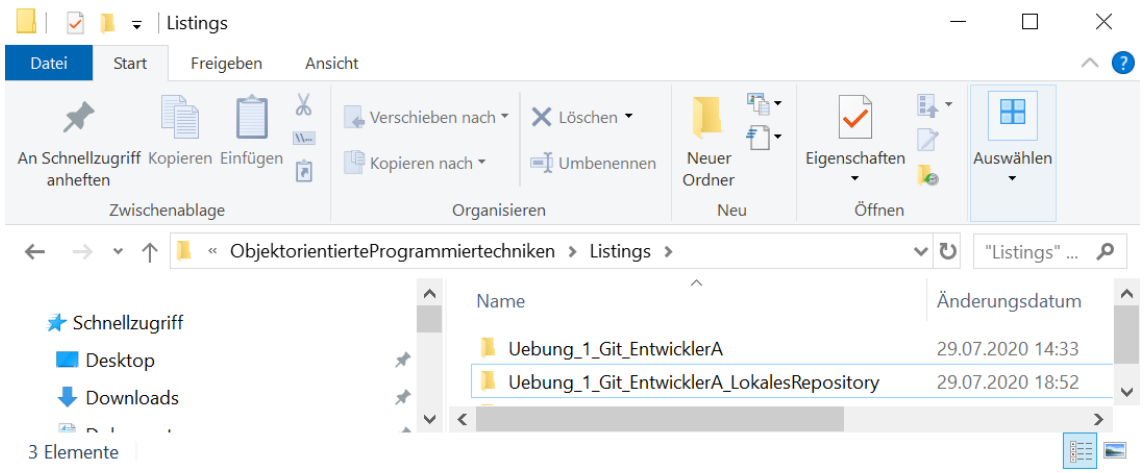
1.2 Durchführung

Erstellen des Projekts

Entwickler A erstellt ein workspace, hier in der Beschreibung *Uebung_1_Git_EntwicklerA,* mit einem Projekt mit Namen *Basisanwendung* mit einem package *gui* und einer *Klasse Main,* welche sich im package *gui* befindet und eine main-Methode enthält, welche die Konsolenausgabe *Guten Tag!* enthält. (Benutzen Sie das jdk.1.8.xxx , falls vorhanden und setzten Sie das Compiler Compliance Level auf 1.8, siehe Kurzanleitung_JavaVersionenPackages!) Ansonsten müssen Sie für die Einbindung von JavaFX zusätzlich die entsprechende Kurzanleitung_JavaFX bearbeiten.)

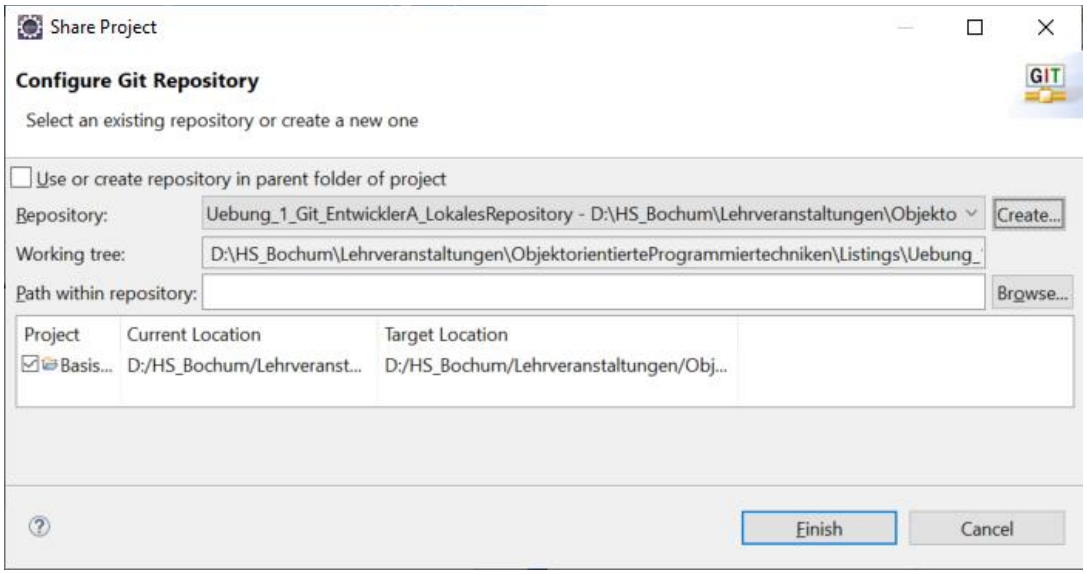
Erstellen eines lokalen Repositories und Anlegen des Projekts im lokalen Repository

Zum Erstellen eines lokalen Repositories erstellt Entwickler A einen neuen Ordner. Für diese Beschreibung wurde ein Ordner *Uebung_1_Git_EntwicklerA_LokalesRepository* angelegt.



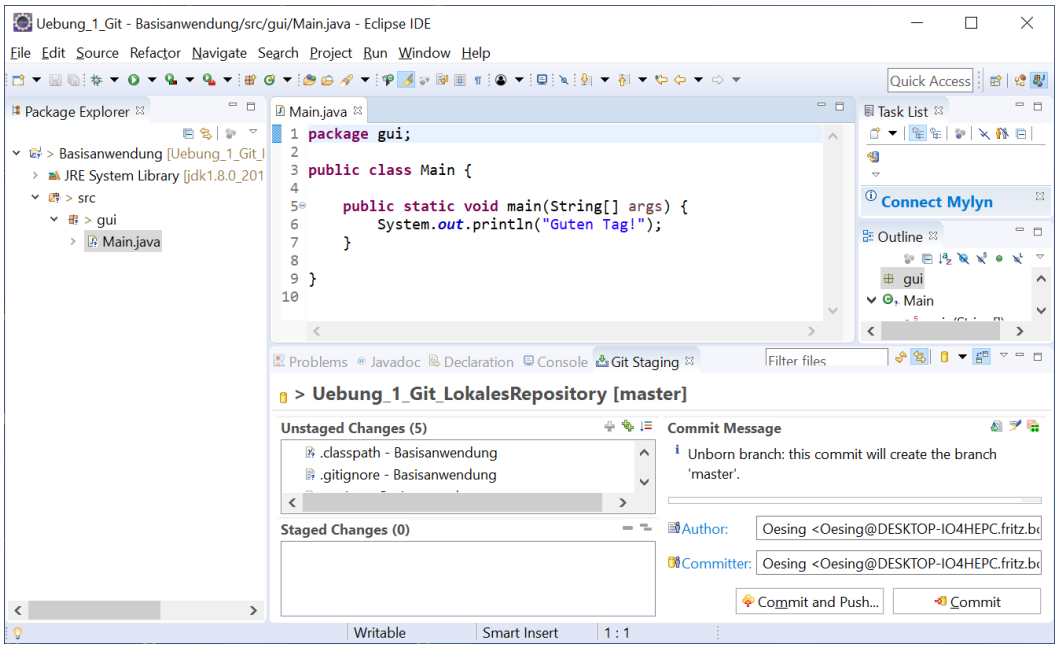
Zur Aufnahme des Projekts in das lokale Repository wählt Entwickler A im Kontextmenü des Projekts *Team / Share Project* aus. In dem dann folgenden Fenster, siehe nächste Seite, gibt er mittels Button *Create…* als Repository denjenigen Ordner an, den er für das lokale Repository erstellt hat, hier *Uebung_1_Git_EntwicklerA_LokalesRepository*. Den *Default branch name* lässt er auf *master* stehen. Er bestätigt die Auswahl mit *Finish (2x).* Wenn man jetzt in den

Ordner des workspaces und in den des lokalen Repositories schaut, wird man feststellen, dass der Projektordner in das lokale Repository verschoben wurde.

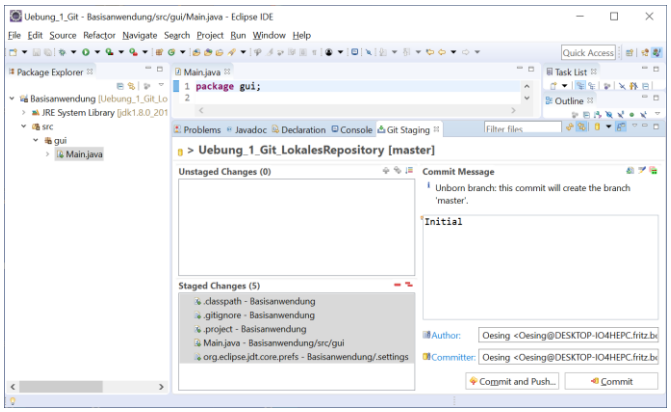


Bearbeiten des Projekts im lokalen Repository

Zum Vorbereiten von Dateien des Projekts für ein *Commit*, also für die Aufnahme von Änderungen des Projekts vom lokalen workspace in das lokale Repository, wählt Entwickler A im Kontextmenü des Projekts *Team/Commit* aus. Er erhält den folgenden Tabreiter *Git Staging*.



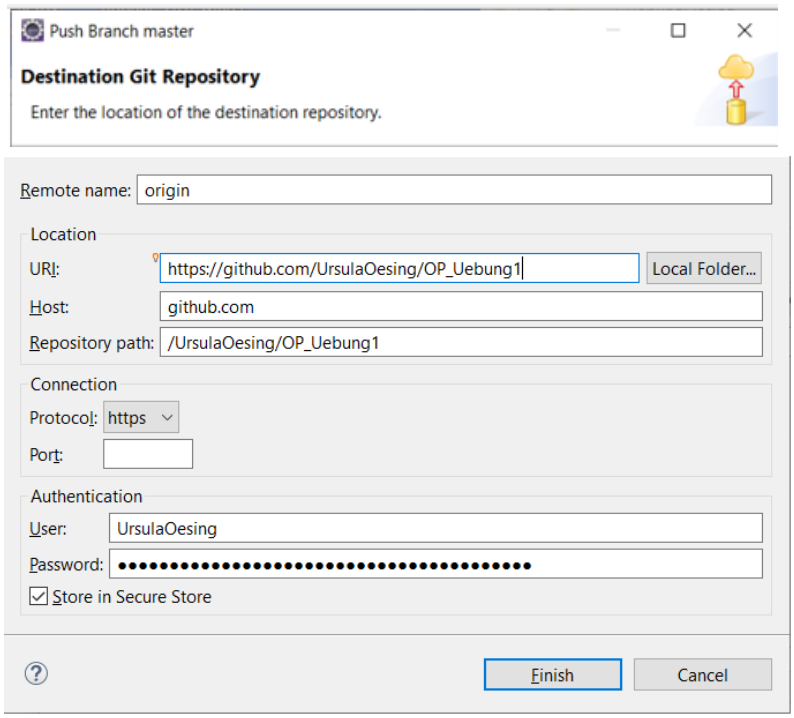
Um unbestätigte Änderungen von *Unstaged Changes* nach *Staged Changes* zu übernehmen, selektiert er die Dateien und schiebt diese mittels drag and drop in das Feld *Staged Changes*.



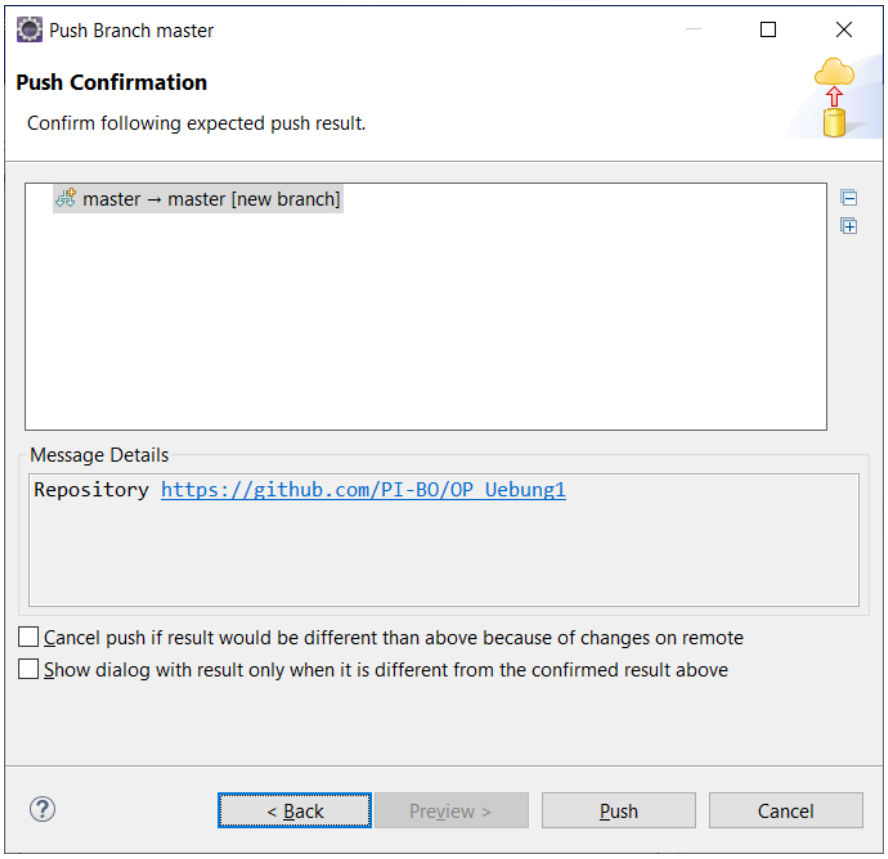
Dann gibt er einen Kommentar ein und bestätigt den Vorgang mittels *Commit.* Nur Dateien aus *Staged Changes* werden *committed* und damit im Repository verwaltet.

Übermitteln des Projekts an das allgemeine Git-Repository

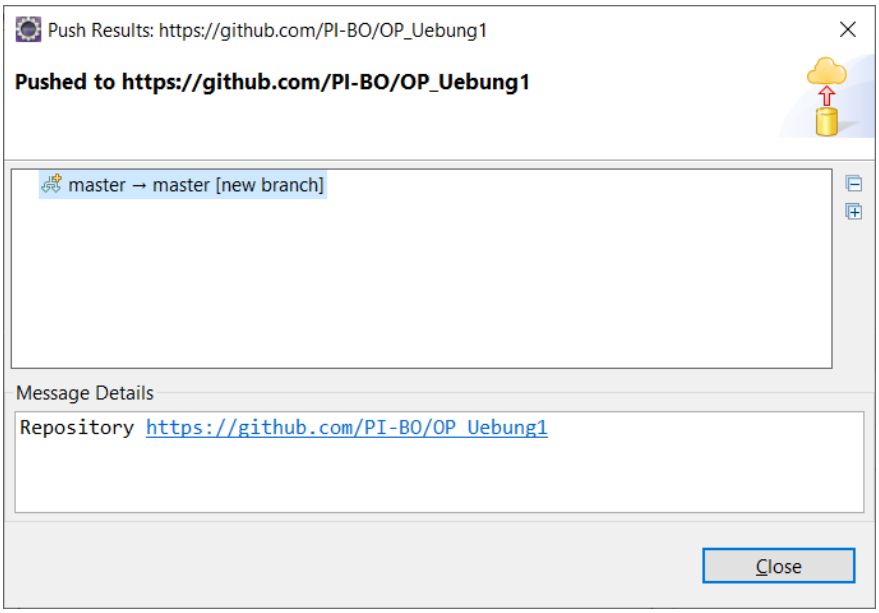
Um die lokalen Änderungen an das allgemeine Git-Repository übermitteln, meldet Entwickler A sich zuerst an diesem an. Dazu öffnet er einen Internet Browser und gibt *https://github.com/NameDesBenutzers/NameDesRepositories* ein. Gegebenenfalls muss er sich mit der Eingabe von *User* und *Password* anmelden. Um die Anmeldung nicht wiederholen zu müssen, klickt er *Store in Secure Store* an. Er bestätigt die Eingaben mit *OK.*
Dann wählt er in Eclipse im Kontextmenü des Projekts *Team/Push Branch 'master'* aus. In dem folgenden Fenster trägt er Daten des GIT Hub's ein und bestätigt diese mit *Preview* und dann nochmals *Preview.*
Achtung: Hier (und gegebenenfalls in dem folgenden Fenster) muss als Password das Access-Token, welches generiert wurde, eingegeben werden, siehe *KurzanleitungZuGit*!



Entwickler A klickt dann auf *Push.* Danach ist das Projekt auch in dem allgemeinen Repository.

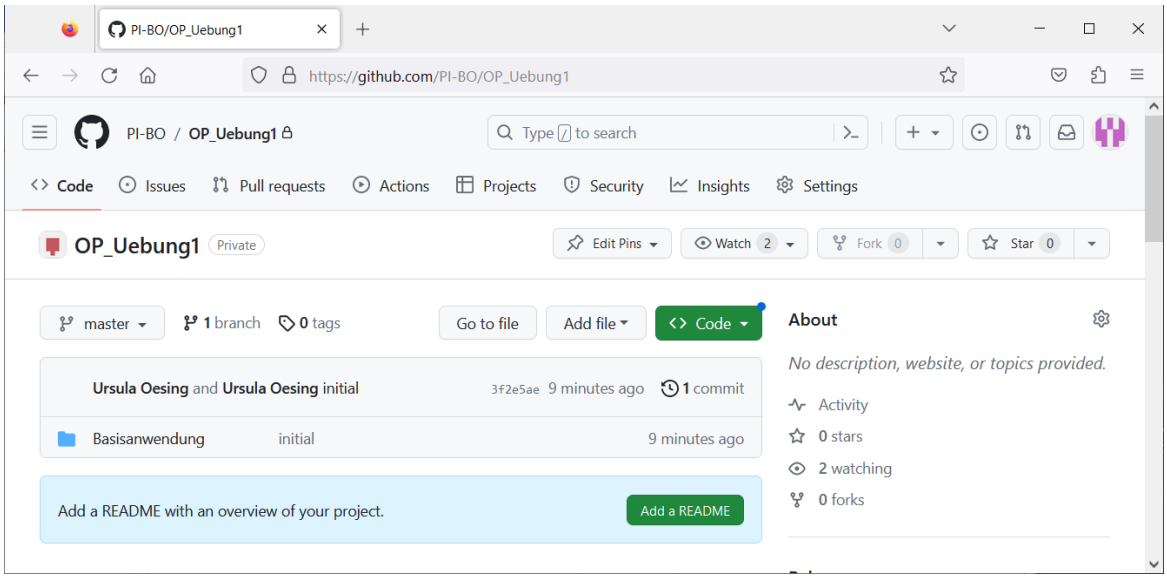


Es wird ein Fenster aufgeblendet, welches den Push-Vorgang bestätigt.

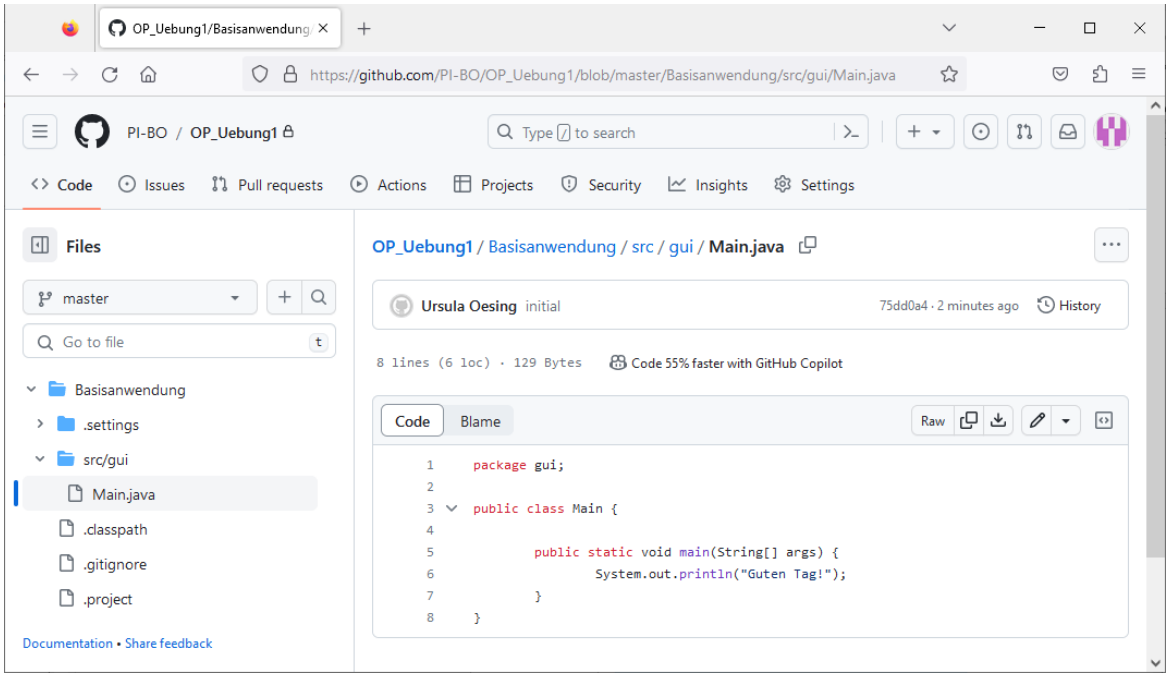


Entwickler A schließt dieses Fenster mittels *Close*.

Hier ist das Projekt in dem allgemeinen Repository zu sehen.



Mit einem Klick auf *Basisanwendung* kann Entwickler A sich die Struktur des Projekts ansehen. Falls die Klasse *Main* noch nicht zu sehen ist, nimmt er in der Klasse lokal eine kleine Änderung vor. Im Tabreiter *Git Staging,* siehe oben, taucht die Klasse dann in der Rubrik *Unstaged Changes* auf. Er schiebt sie nach *Staged Changes,* siehe oben, gibt einen Kommentar ein und transferiert sie mittels *Commit and Push* in das lokale und auch allgemeine Repository.

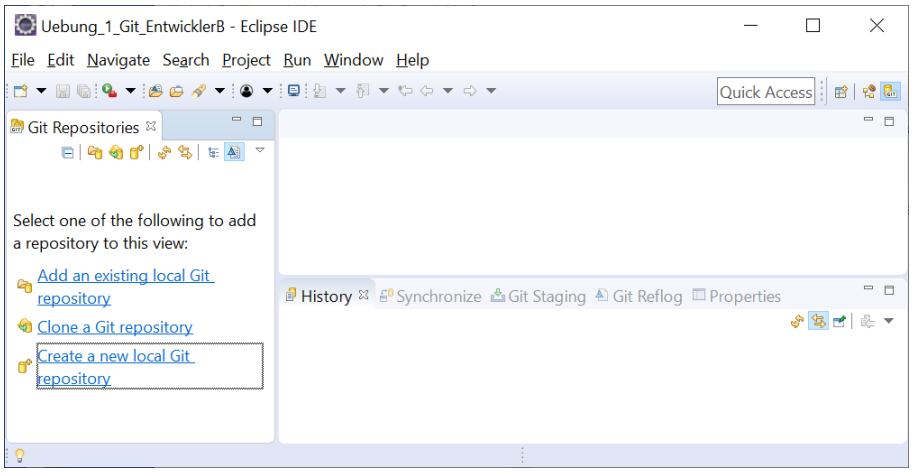


Dann beendet er Eclipse. (Vorstellung: Entwickler A hat Feierabend und geht nach Hause.)

Anschluss eines bestehenden Projekts aus einem allgemeinen Git Repository an ein lokales Repository

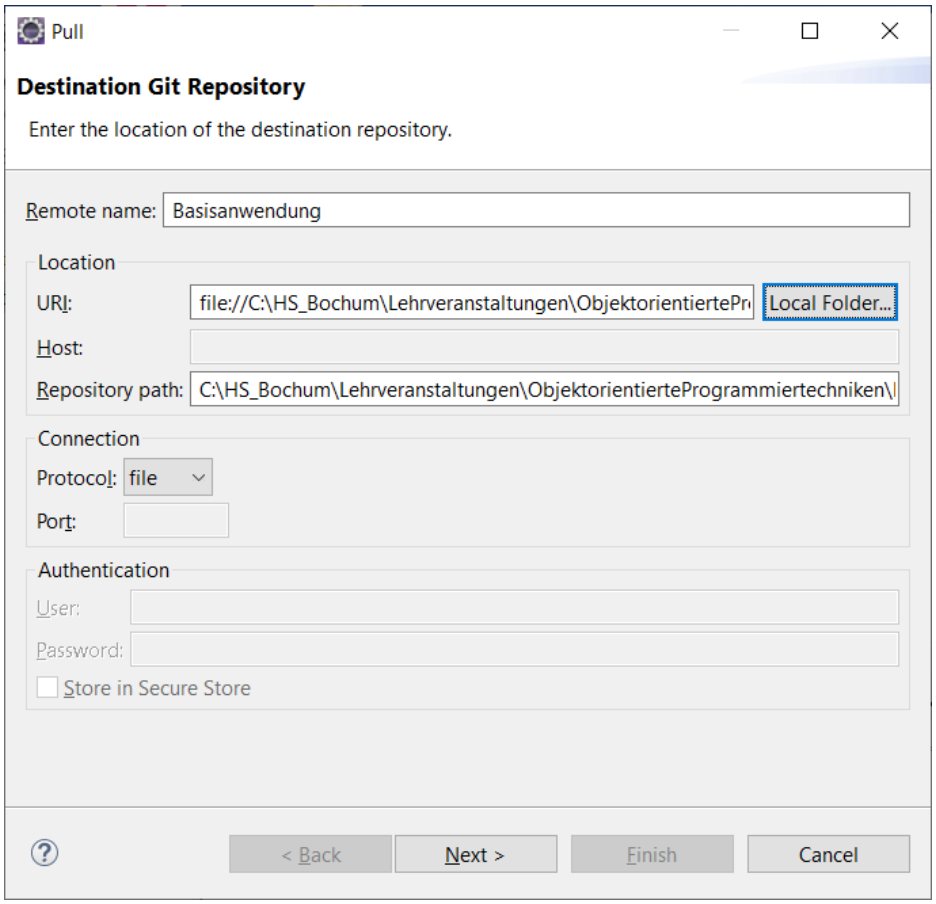
Entwickler A und B beginnen am neuen Arbeitstag mit der Realisierung ihrer jeweiligen Aufgabe. Entwickler A soll die Klasse *Anwendungssystem* erstellen und die Klasse *Main* weiterentwickeln, Entwickler B soll die Klasse *CsvDateiLeser* entwickeln.

Dazu holen beide den aktuellen Projektstand des Git-Repositories komplett auf ihren Client-Rechner (auschecken):
Bei Entwickler A ist das Projekt bereits vorhanden. (Falls nicht, da zwischendurch alles gelöscht wurde, muss er vorgehen wie Entwickler B.)
Bei Entwickler B ist das Projekt noch nicht vorhanden. Er meldet sich in GitHub an. Dann erstellt er mittels Eclipse ein workspace, hier *Uebung_1_Git_EntwicklerB,* und legt ein Verzeichnis für ein lokales Repository an, hier *Uebung_1_Git_EntwicklerB_LokalesRepository.* Er wechselt die Perspektive in Eclipse mit dem Icon *Open Perspective* rechts oben oder mittels *Window/Perspective/Open Perspective/Other.* Dann wählt er *Git* aus. Im Tabreiter *Git Repositories* wählt er *Create a new local Git repository* aus.

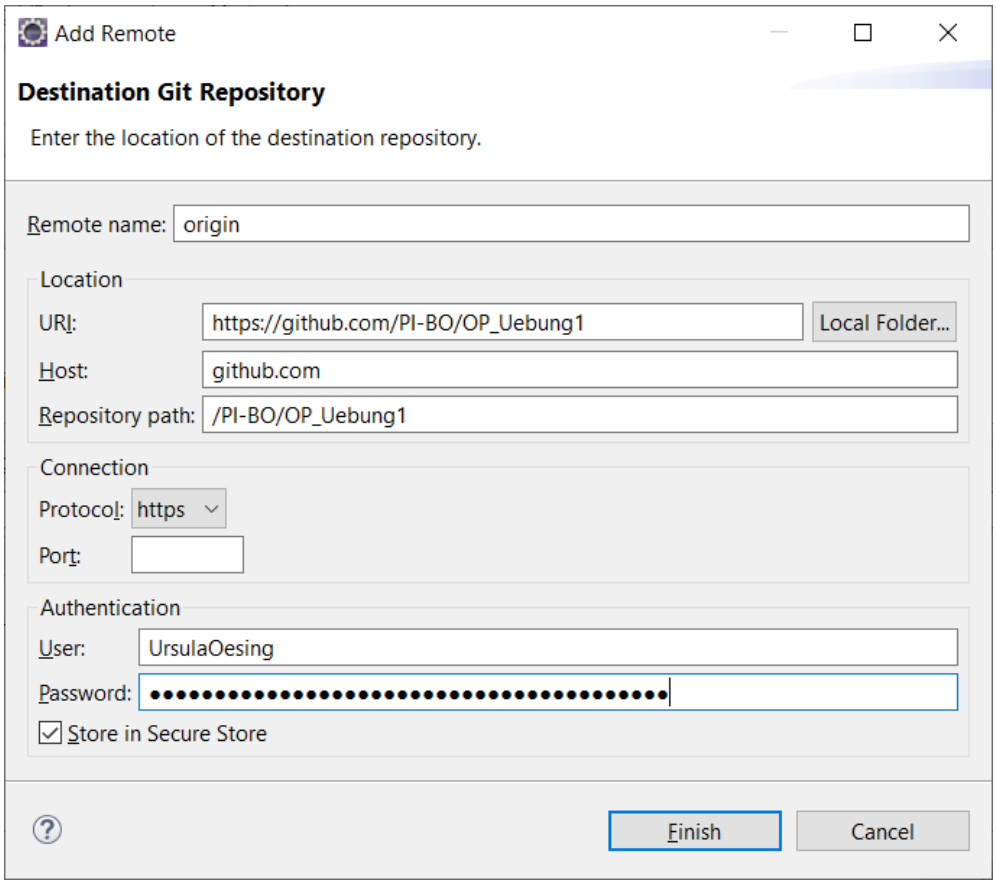


In dem dann folgenden Fenster gibt er mittels *Browse…*als Repository directory das neue Verzeichnis, hier *Uebung_1_Git_EntwicklerB_LokalesRepository,* an und klickt auf *Create.*

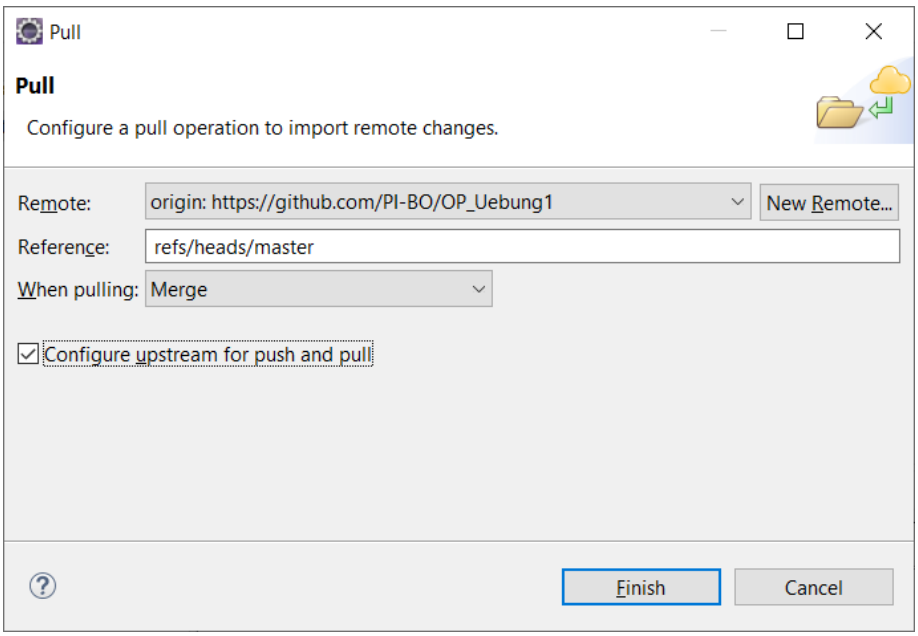
Um das Projekt aus dem allgemeinen Git Repository zu holen, wählt er im Kontextmenü seines lokalen Repositories *Pull…* aus. In dem dann folgenden Fenster gibt er als *Remote name Basisanwendung* ein und wählt als *URI* sein lokales Repository an und bestätigt dieses mit *Next >* .



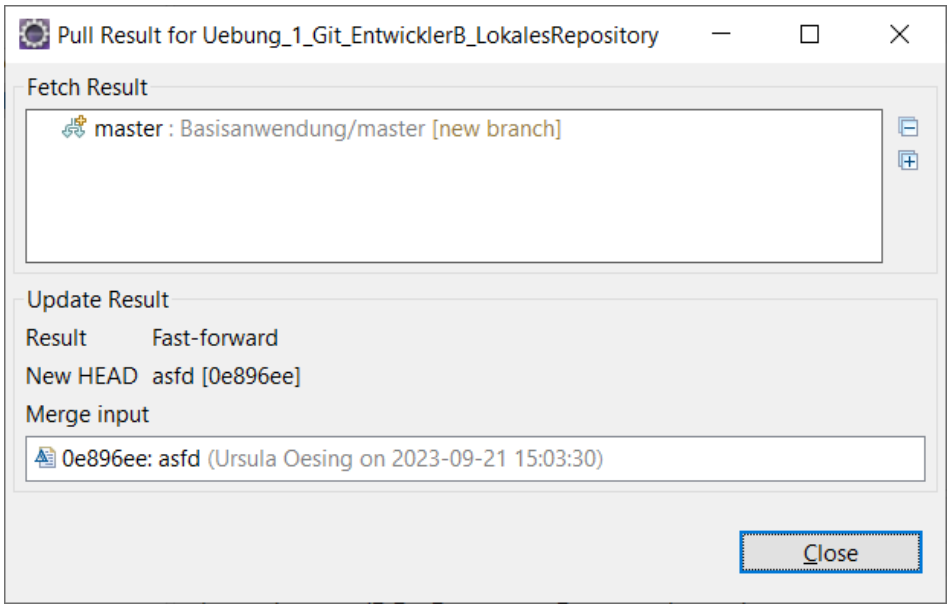
Dann wählt er in dem folgenden Fenster *New Remote…* aus und gelangt in das Fenster zum Eingeben der Daten des allgemeinen GIT Hub's.



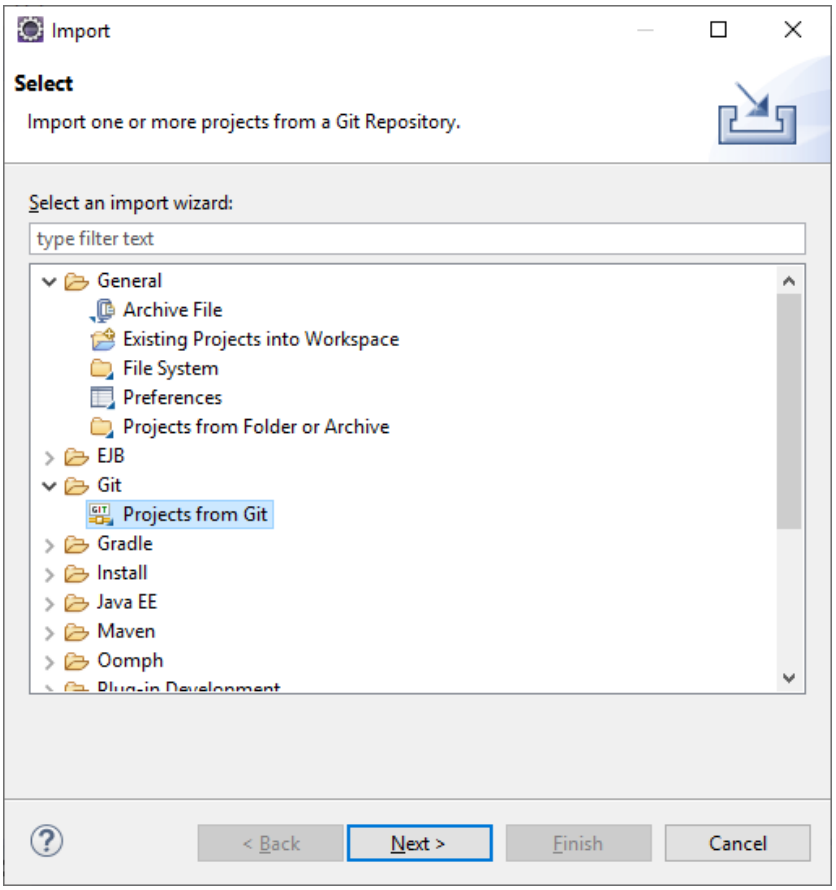
Achtung: Auch hier muss als Password das Access-Token, welches generiert wurde, siehe *KurzanleitungZuGit,* eingegeben werden!
Er bestätigt die Eingaben mit *Finish.* Das Remote wird übernommen. Dann gibt er als Referenz *refs/heads/master* ein, setzt einen Haken bei *Configure upstream for push and pull* und bestätigen die Eingabe mit *Finish.*



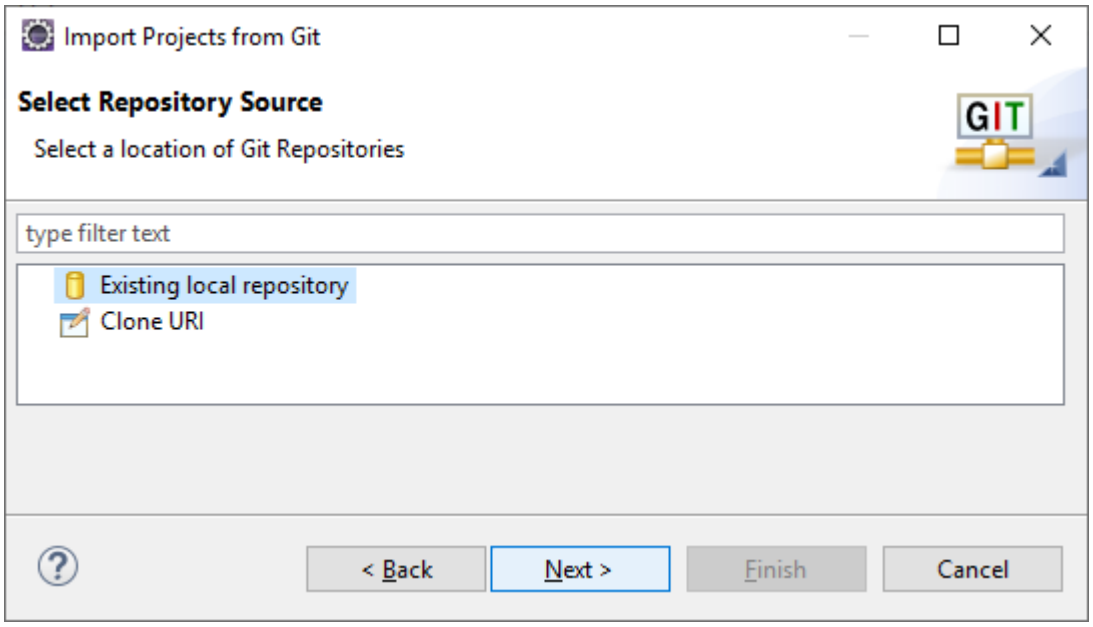
Der Inhalt des allgemeinen Repositories wird übernommen.



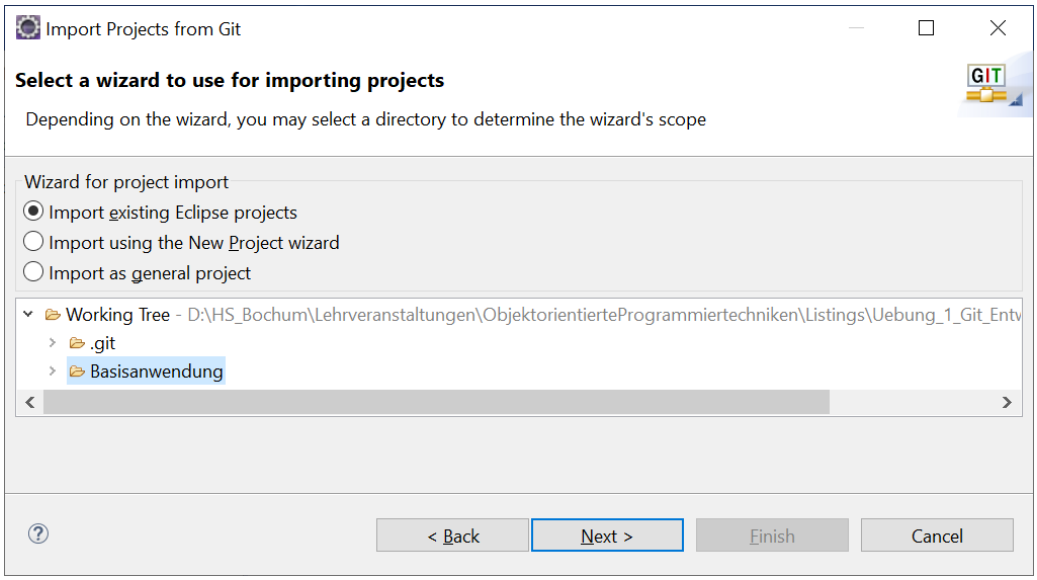
Danach wechselt er in die Java Perspektive. Das Projekt muss jetzt noch importiert werden. Dazu wählt er *File/Import…* aus. In dem dann folgenden Fenster wählen er *Git/Projects from Git* aus und bestätigen Ihre Auswahl mit *Next>.*



Dann wählt er *Existing local repository* aus und bestätigt die Auswahl mit *Next >.*



In dem dann folgenden Fenster selektiert er das Git Repository und klickt auf *Next>.*
In dem dann folgenden Fenster wählt er das zu importierende Projekt aus und klickt auf *Next>.*



In den dann folgenden Fenstern wählt er *Next* > bzw. *Finish* aus. Das Projekt des allgemeinen Repositories wird übernommen.

Auschecken / Einchecken des Projekts

Entwickler A legt im package *gui* die Klasse *Anwendungssystem* an. Die rot markierte Zeilen muss er noch weglassen, da das Programm sonst nicht kompilierbar ist.

Klasse Anwendungssystem

```
package gui;

import java.io.*;
import business.CsvDateiLeser;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.*;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.Alert.AlertType;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.text.*;
import javafx.stage.Stage;

/**
 * erstellt das View zum Basisfenster
 */
public class Anwendungssystem{

        // Objekt zum Lesen der Kunden.csv - Datei
        private CsvDateiLeser csvDateiLeser;
```

```
private GridPane grid        = new GridPane();
private Label lblAnzeige     = new Label ("Anzeige");
private TextArea txtAnzeige  = new TextArea();
private Button btnAnzeige    = new Button("Lesen und Anzeigen");

/**
* erstellt das View zum Basisfenster inklusive Titel,
* Komponenten und Listener und zeigt es an.
* @param primaryStage das Stage-Objekt zum Basisfenster
*/
public Anwendungssystem(Stage primaryStage){
      this.csvDateiLeser = new CsvDateiLeser();
      this.grid.setAlignment(Pos.CENTER);
      this.grid.setHgap(10);
      this.grid.setVgap(10);
      this.grid.setPadding(new Insets(25, 25, 25, 25));
      primaryStage.setTitle(
            this.csvDateiLeser.getUeberschrift());
      Scene scene = new Scene(grid, 210, 200);
      primaryStage.setScene(scene);
      primaryStage.show();
      this.initKomponenten();
      this.initListener();
}

/* initialisiert die Steuerelemente auf der Maske */
private void initKomponenten(){
      grid.add(lblAnzeige, 1, 0);
      lblAnzeige.setFont(Font.font("Arial", FontWeight.BOLD, 24));
      grid.add(txtAnzeige, 1, 1);
      txtAnzeige.setMaxSize(170, 100);
      txtAnzeige.setBackground(new Background(
            new BackgroundFill(Color.YELLOW, CornerRadii.EMPTY,
            Insets.EMPTY)));
      this.grid.add(btnAnzeige, 1, 2);
       btnAnzeige.setMaxSize(170, 25);
}

/* initialisiert die Listener zu den Steuerelementen auf der Maske
*/
private void initListener(){
}

 /* zeigt den Inhalt des Arrays zeilen in txtAnzeige an. */
private void zeigeAn(String[] zeilen){
      if(zeilen != null){
            for(int i = 0; i < zeilen.length; i++){
                  txtAnzeige.insertText(
                        txtAnzeige.getText().length(),
                  zeilen[i] + "\n");
            }
      }
}
```

```
	/** zeigt ein Fehlermeldungsfenster an
	* @param meldung, String, welcher die Fehlermeldung enthaelt
	*/
	public void zeigeFehlermeldung(String ueberschrift,
		String meldung){
		Alert alert = new Alert(AlertType.ERROR);
		alert.setTitle("Fehlermeldung");
		alert.setHeaderText(ueberschrift);
		alert.setContentText(meldung);
		alert.show();
	}
}
```

Die Klasse *Main* ändert er jetzt folgendermaßen ab.

Klasse Main

```
package gui;

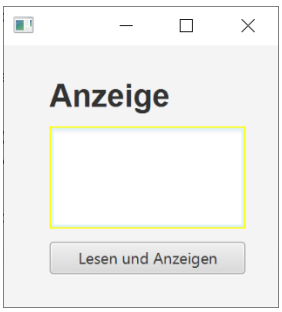
import javafx.application.Application;
import javafx.stage.Stage;


public class Main extends Application {

	@Override
	public void start(Stage primaryStage) {
		new Anwendungssystem(primaryStage);
	}

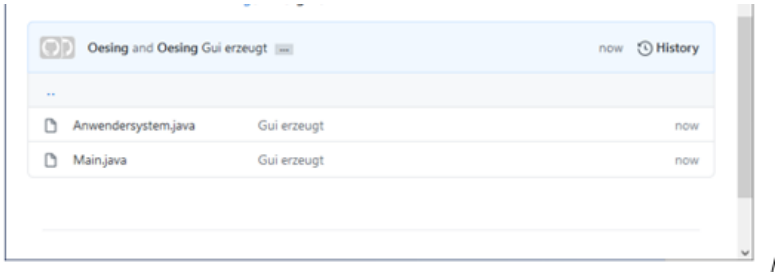
	public static void main(String[] args){
		launch(args);
	}
}
```

Entwickler A kann seine geänderte Anwendung ausführen. Allerdings passiert noch nicht beim Klick auf den Button *Lesen und Anzeigen.*



Dann checked den neuen Stand des Projekts in sein lokales Repository ein, indem er im Kontextmenü des Projekts *Team / Commit* aussucht und in dem folgenden Fenster wie bereits beschrieben vorgeht.

Dann wählt er im Kontextmenü des Projekts *Team / Push Branch 'master'* aus, um die Änderung an das Git-Repository zu senden. Er bestätigt mit *Preview >* und *Push >.* Er erhält als Preview wie oben ein Fenster mit einer Bestätigung, dass gepushed werden kann. Alternativ kann Entwickler A im Commit-Fenster anstatt auf *Commit* auf *Commit and Push…*klicken.
Entwickler A kann sich im Internet-Browser die Änderung ansehen.



Entwickler B legt das package *business* und dort die Klasse *CsvDateiLeser* an.

Klasse CsvDateiLeser

```
package business;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Vector;

/**
 * Klasse, welche Kunden.csv liest
 */
 public class CsvDateiLeser {

       /**
        * Gibt die Ueberschrift fuer das Grundfenster heraus
        * @return String, Ueberschrift fuer das Grundfenster
        */
        public String getUeberschrift(){
              return "Hallo";
        }

       /**
        * liest die Datei Kunden.csv und gibt sie als String- Array
        * zurueck.
        * @return String[], enthaelt die Zeilen der Datei Kunden.csv
        * @throws FileNotFoundException,
                  wird geworfen, wenn csv-Datei nicht vorhanden
        * @throws IOException, wird geworfen, wenn ein Fehler beim Lesen
        *                      auftaucht
        */
        public String[] leseKunden()
                    throws FileNotFoundException, IOException{
```

```
            Vector<String> ergebnis = new Vector<String>();
                  String zeile = null;
            BufferedReader ein
                  = new BufferedReader(new FileReader("Kunden.csv"));
            zeile = ein.readLine();
            while(zeile != null && zeile.length() != 0){
                  ergebnis.add(zeile);
                  zeile = ein.readLine();
            }
            ein.close();
            return ergebnis.toArray(new String[]{});
      }
}
```

Aus Testzwecken ändert Entwickler B die Klasse *Main* folgendermaßen ab.

Klasse Main

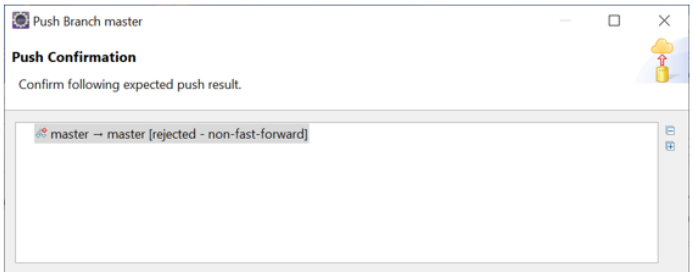
```
package gui;

import business.CsvDateiLeser;

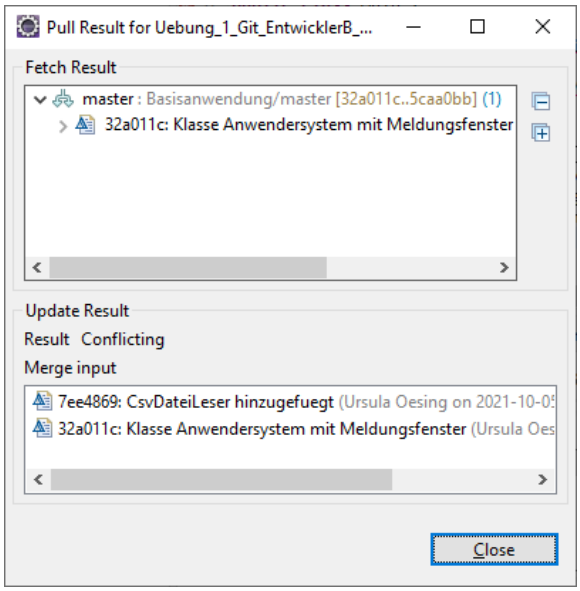
public class Main {

      public static void main(String[] args) {
            CsvDateiLeser cdl = new CsvDateiLeser();
            String ueberschrift = cdl.getUeberschrift();
            System.out.println(ueberschrift);
      }
}
```

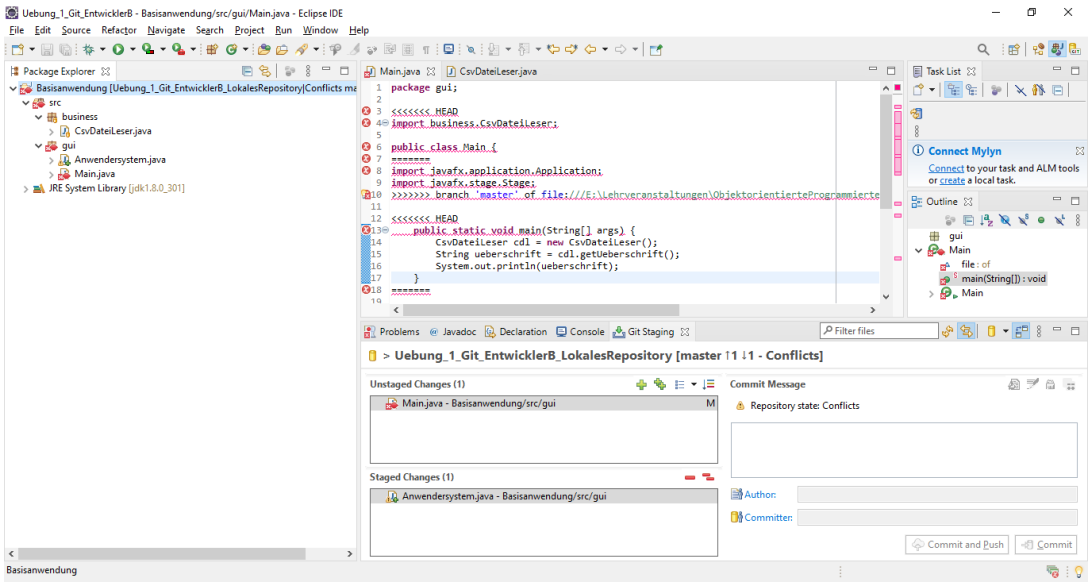
Entwickler B testet die Anwendung. Danach versucht er, die neue Klasse *CsvDateiLeser* und die geänderte Klasse *Main* einzuchecken. Die Übermittlung in sein lokales Repository gelingt, nicht aber die in das allgemeine Git Repository.



Er muss erst den aktuellen Stand vom allgemeinen Git Repository holen, da dieses sich geändert hat. Das macht er mittels *Team/pull.*



Problem ist die Klasse *Main,* in welcher es Konflikte gibt. Die neue Klasse *Anwendungssystem* führt nicht zu Konflikten.



Entwickler B führt den bisherigen Stand zu einer lauffähigen Anwendung zusammen. Er übernimmt die Klasse *Main* von Entwickler A und kommentiert die roten Zeilen der Klasse *Anwendungssystem* wieder ein. Das Ergebnis pushed er zum allgemeinen Git Repository.

Entwickler A updated seinen Projektstand mittels *Team / pull*. Er ergänzt die Klassen *Anwendungssystem* folgendermaßen.

```
/* initialisiert die Listener zu den Steuerelementen auf der Maske
*/
private void initListener(){
    btnAnzeige.setOnAction(new EventHandler<ActionEvent>() {
```

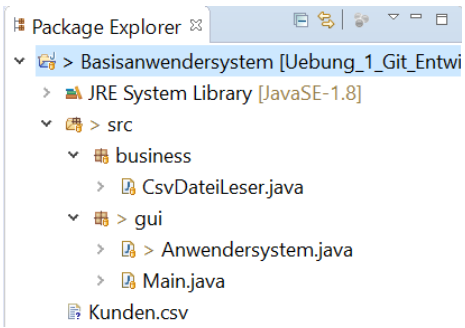
```
            @Override
            public void handle(ActionEvent e) {
                String[] ergebnis = leseKunden();
                zeigeAn(ergebnis);
            }
        });
    }

/*
 * liest die Zeilen der Datei Kunden.csv und gibt sie als
 * String-Array zurueck.
 * @return String[], enthaelt die Zeilen der Datei Kunden.csv
 */
private String[] leseKunden(){
      String[] ergebnis = null;
      try{
          ergebnis = this.csvDateiLeser.leseKunden();
      }
      catch(FileNotFoundException fnfExc){
          this.zeigeFehlermeldung("FileNotFoundException",
                "Datei wurde nicht gefunden!");
      }

      catch(IOException ioExc){
          this.zeigeFehlermeldung("IOException",
              "Fehler beim Lesen der Datei!");
      }
      catch(Exception exc){
          this.zeigeFehlermeldung("Exception",
              "Unbekannter Fehler!");
      }
      return ergebnis;
}
```

Weiterhin erstellt Entwickler A im Projektordner eine csv-Datei *Kunden.csv* über *New/File* im Kontextmenü des Projekts. Diese enthält den folgenden Inhalt:

```
Hans;Müller
Helmut;Meier
```



Jetzt ist die Anwendung lauffähig und erfüllt die gewünschte Funktionalität. Beim Klick auf *Lesen und Anzeigen* wird die csv-Datei gelesen und der Inhalt angezeigt.



Entwickler A checked den Stand des Projekts inklusive csv-Datei ein mittels *Team/Commit,* drag an drop der *Unstaged Changes* zu den *Staged Changes,* Eingabe eines Kommentars und dann *Commit and Push*… . . Er kann sein Einchecken kontrollieren, indem er im Internet Browser nachsieht, ob *Kunden.csv* vorhanden ist.
Entwickler B holt sich den aktuellen Stand mittels *Team/Pull.* Er ändert dann die Schriftgröße des Labes *Anzeige* in der Klasse *Anwendungssystem* von 24 auf 20 und checked die Veränderung wieder ein.
Entwickler A hingeben verändert die Schriftgröße des Labels *Anzeige* in der Klasse *Anwendungssystem* von 24 auf 28, ohne sich vorher die letzte Änderung von Entwickler A zu holen. Versucht Entwickler A, diese Änderung auf das allgemeine Repository einzuchecken, erhält er eine Fehlermeldung, siehe oben.
Auch er muss erst den aktuellen Stand vom allgemeinen Git Repository mittels *Team/pull* holen, da dieses sich geändert hat.

```
    /* initialisiert die Steuerelemente auf der Maske */
    private void initKomponenten(){
        grid.add(lblAnzeige, 1, 0);
<<<<<<< HEAD
        lblAnzeige.setFont(Font.font("Arial", FontWeight.BOLD, 28));
=======
        lblAnzeige.setFont(Font.font("Arial", FontWeight.BOLD, 20));
>>>>>>> branch 'master' of https://github.com/PI-BO/ObjektorientierteProgrammierung
        grid.add(txtAnzeige, 1, 1);
        txtAnzeige.setMaxSize(170, 100);
        txtAnzeige.setBackground(new Background(
            new BackgroundFill(Color.YELLOW, CornerRadii.EMPTY,
            Insets.EMPTY)));
        this.grid.add(btnAnzeige, 1, 2);
        btnAnzeige.setMaxSize(170, 25);
    }
```

Entwickler A nimmt sich das Mergen vor, indem er die Schriftgröße wieder auf 24 setzt, also den Ausgangswert.
Entwickler B holt die aktuelle Änderung nicht, ändert in der Klasse *CsvDateiLeser* den return-Wert von *getUeberschrift()* auf *HalliHallo.* Das Einchecken der Klasse *CsvDateiLeser* wird gestattet. Wird also eine Änderung in einer anderen Klasse vorgenommen, so wird diese grundsätzlich erlaubt.